Praktische Arbeit WMH

Inhaltsverzeichnis

Einleitung 2
Planung 2
Technologien 3
ERD-Beschreibung 3
Erklärung des SQL-Skripts 5
Resultat 6
Reflexion 7

Einleitung

Im Rahmen dieses Projekts wird eine Webplattform als öffentlicher Media Host entwickelt, auf der Fotos, Videos und Sounds organisiert, verwaltet und zugänglich gemacht werden. Die Medien werden in einem GitHub-Repository gespeichert, nach Kategorien gegliedert und mit aussagekräftigen Signaturen versehen. Die Plattform bietet eine Filterfunktion nach Medientyp und Kategorie, eine effektive Suchfunktion sowie die Möglichkeit, Dateien direkt über die GitHub API hochzuladen und zu löschen.

Unsere Gruppe heisst Exodia. Es besteht aus 7 Personen, und zwar aus Tim, Kevin, Eric, Danylo, Denis, Ottavio und Akash. Unsere Domain heisst https://kepap.tim-thiele.workers.dev/

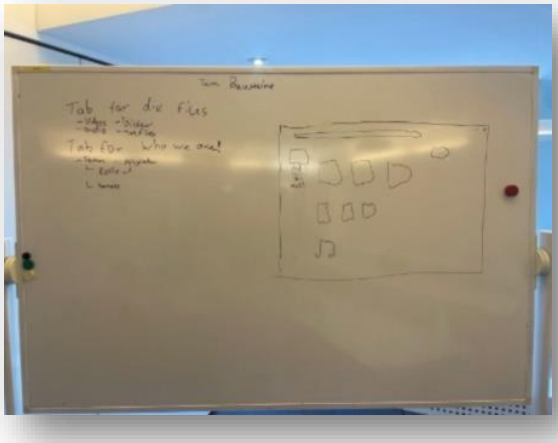
Planung

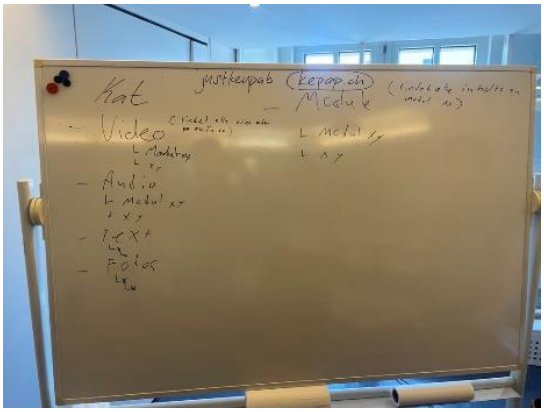
Zu Beginn des Projekts wurde die Struktur der Webseite geplant. Die Plattform soll übersichtlich aufgebaut sein und dem Benutzer eine einfache Navigation ermöglichen. Die Homepage bildet dabei den zentralen Einstiegspunkt, von dem aus den verschiedenen Inhalten erreichbar sind.

Die Webseite wurde in unterschiedliche Kategorien wie Audio, Text, Video und Fotos unterteilt. Diese Kategorien sind über eine Seitenleiste (Sidebar) zugänglich, sodass der Benutzer schnell zwischen den Inhalten wechseln kann.

Zusätzlich wurde geplant, eine Benutzerverwaltung zu integrieren. Im Datenbankschema wurden Tabellen für Benutzer, Rollen (Administrator, Editor, Viewer, Uploader) und Teams vorbereitet. Die Authentifizierung für Upload und Löschfunktionen erfolgt aktuell über GitHub Personal Access Tokens.

Ein weiterer wichtiger Bestandteil der Planung war die Möglichkeit, Medien hochzuladen und in einem GitHub-Repository zu speichern. Dabei wurde darauf geachtet, dass die Dateien strukturiert in Kategorie-Ordnern abgelegt und automatisch den entsprechenden Kategorien zugeordnet werden.





Technologien

Für die Entwicklung unserer Media-Hosting-Plattform verwenden wir verschiedene Webtechnologien.

HTML wird verwendet, um die Struktur der Webseite zu erstellen, zum Beispiel Überschriften, Texte, Bilder und Formulare. CSS sorgt für das Design und das Layout der Seite, wie Farben, Schriftarten und Abstände.

Für interaktive Funktionen im Browser verwenden wir JavaScript, zum Beispiel für Navigation, Buttons, Suchfunktion, Datei-Upload und die Kommunikation mit der GitHub API. Die gesamte Anwendung läuft clientseitig im Browser – es wird kein serverseitiges Backend wie PHP benötigt. Stattdessen werden Dateien direkt über die GitHub API gelesen, hochgeladen und gelöscht.

Die MySQL-Datenbank (kepap.sql) enthält das konzeptionelle Schema für die Medienverwaltung mit Tabellen für Kategorien, Benutzer, Rollen, Teams und Medien. Dieses Schema bildet die Grundlage für eine zukünftige Erweiterung mit einem vollständigen Backend.

Zusätzlich verwenden wir Cloudflare Workers, um die Website schneller und sicherer zu machen. Cloudflare bietet Schutz vor Angriffen, verbessert die Performance durch Caching und hilft dabei, die Website weltweit stabil verfügbar zu machen.

ERD-Beschreibung

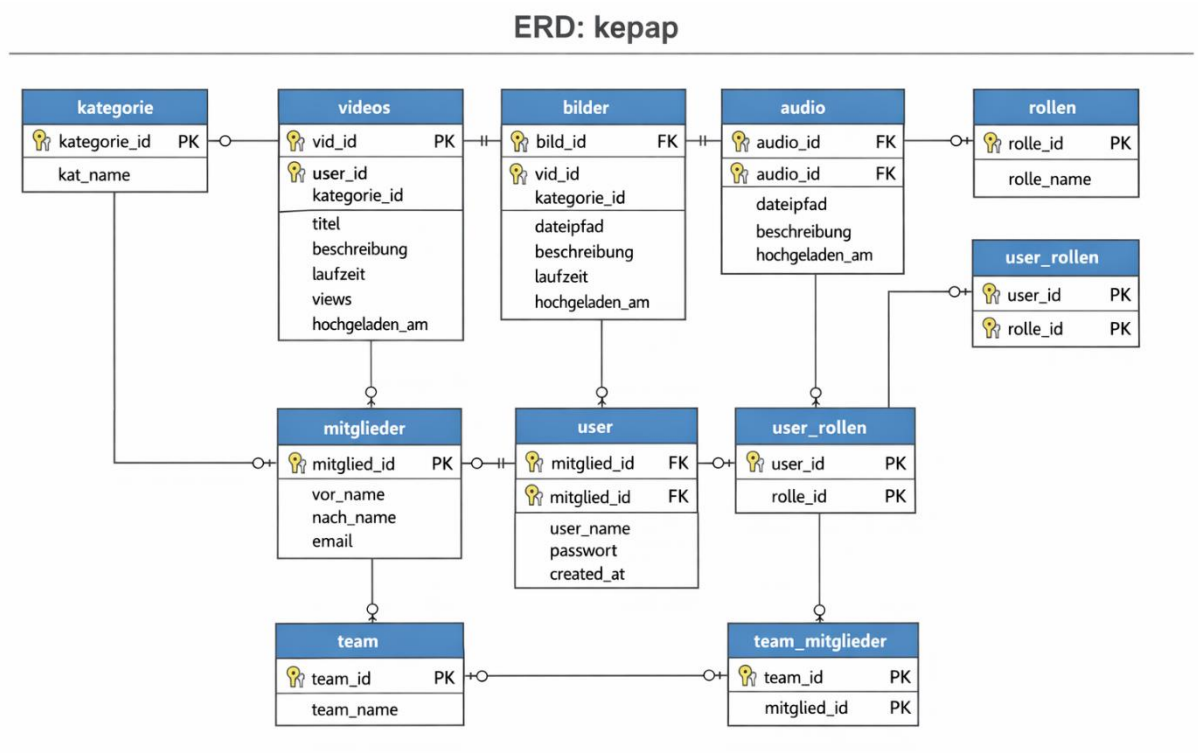
Die Tabelle *kategorie* enthält die 19 Schulkategorien (z.B. IFZK-2524-017-S2-UEK-106), die wir zur besseren Organisation der Medien eingeführt haben. Medien wie Videos, Bilder und Audio können jeweils einer Kategorie zugeordnet werden. In den Medientabellen werden wichtige Informationen wie Titel, Beschreibung, Dateipfad, Laufzeit und Upload-Datum gespeichert.

Die Tabelle *mitglieder* enthält persönliche Informationen der Nutzer, während die Tabelle *user* die Login-Daten wie Benutzername und Passwort verwaltet. Dadurch haben wir eine klare Trennung zwischen persönlichen Daten und Zugangsdaten umgesetzt. Zusätzlich haben wir Rollen integriert, damit Benutzer unterschiedliche Rechte erhalten können, zum Beispiel Admin, Editor, Viewer oder Uploader. Über die Tabelle *user_rollen* werden die Benutzer mit ihren Rollen verbunden.

Ausserdem gibt es die Tabellen *team* und *team_mitglieder*, um mehrere Mitglieder einem Team zuordnen zu können.

Insgesamt ermöglicht das ERD eine strukturierte und flexible Verwaltung von Medien, Kategorien, Benutzern und Teams innerhalb unserer Plattform.

(Dieser Teil war schon korrekt, nur die Kategorien-Bezeichnung angepasst)




```
MariaDB [kepap]> SELECT * FROM kategorie;
+--------------+------------+
| kategorie_id | kat_name   |
+--------------+------------+
|            1 | Technik    |
|            2 | Natur      |
|            3 | Wirtschaft |
+--------------+------------+
```

Erklärung des SQL-Skripts

https://github.com/Kralix1694/Kepap/blob/main/kepap.sql

Das SQL-Skript *kepap.sql* wird verwendet, um die komplette Datenbank für unsere Medienverwaltungsanwendung automatisch zu erstellen. Am Anfang werden vorhandene Tabellen gelöscht, damit das Skript problemlos mehrfach ausgeführt werden kann, ohne dass Fehler entstehen. Dafür werden die Fremdschlüsselprüfungen kurzzeitig deaktiviert und danach wieder eingeschaltet.
Anschließend werden die wichtigsten Tabellen angelegt. Dazu gehören Kategorien, Rollen, Mitglieder, Benutzer und Teams. In der Tabelle *mitglieder* werden persönliche Daten wie Vorname, Nachname und E-Mail gespeichert. Die Tabelle *user* enthält die Login-Daten und ist über einen Fremdschlüssel mit den Mitgliedern verbunden. Kategorien helfen dabei, die Medien zu ordnen, während Rollen festlegen, welche Rechte ein Benutzer hat.
Außerdem werden sogenannte Beziehungstabellen wie *user_rollen* und *team_mitglieder* erstellt. Diese ermöglichen sogenannte n:m-Beziehungen. Das bedeutet, dass ein Benutzer mehrere Rollen haben kann und ein Team aus mehreren Mitgliedern bestehen kann. Dafür werden zusammengesetzte Primärschlüssel verwendet.
Danach folgen die Medientabellen *videos*, *bilder* und *audio*. In diesen Tabellen werden Informationen zu den jeweiligen Medien gespeichert, zum Beispiel Titel, Beschreibung, Dateipfad und das Upload-Datum. Über Fremdschlüssel sind diese Tabellen mit Benutzern, Kategorien oder Videos verbunden, sodass alles logisch zusammenhängt.
Ein wichtiger Teil des Skripts sind die Fremdschlüsselregeln. Mit Optionen wie *CASCADE*, *RESTRICT* und *SET NULL* wird festgelegt, was passiert, wenn Daten geändert oder gelöscht werden. Dadurch bleibt die Datenbank strukturiert und es entstehen keine ungültigen Verknüpfungen.
Zum Schluss werden noch Beispieldaten eingefügt, darunter verschiedene Kategorien und Rollen wie Admin, Editor, Viewer und Uploader. Diese helfen dabei, die Datenbank direkt zu testen und erste Funktionen auszuprobieren.

```sql
-- ---------- kategorie ----------
CREATE TABLE kategorie (
    kategorie_id INT AUTO_INCREMENT PRIMARY KEY,
    kat_name     VARCHAR(60) NOT NULL UNIQUE
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;

-- ---------- rollen ----------
CREATE TABLE rollen (
    rolle_id   INT AUTO_INCREMENT PRIMARY KEY,
    rolle_name VARCHAR(50) NOT NULL UNIQUE
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;

-- ---------- mitglieder ----------
CREATE TABLE mitglieder (
    mitglied_id INT AUTO_INCREMENT PRIMARY KEY,
    vor_name    VARCHAR(80) NOT NULL,
    nach_name   VARCHAR(80) NOT NULL,
    email       VARCHAR(120) NOT NULL UNIQUE
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;
```

In MariaDB wurde mit dem Befehl SHOW INDEX FROM kunde, eine Übersicht aller Indizes der Tabelle angezeigt. Jeder Eintrag beschreibt dabei einen Index oder einen Teil eines zusammengesetzten Indexes.

```sql
-- ---------- videos ----------
CREATE TABLE videos (
    vid_id         INT AUTO_INCREMENT PRIMARY KEY,
    user_id        INT           DEFAULT NULL,
    kategorie_id   INT           DEFAULT NULL,
    titel          VARCHAR(200) NOT NULL,
    beschreibung   TEXT,
    laufzeit       INT           DEFAULT 0 COMMENT 'Sekunden',
    views          INT           DEFAULT 0,
    hochgeladen_am DATETIME      NOT NULL DEFAULT CURRENT_TIMESTAMP,
    CONSTRAINT fk_vid_user
        FOREIGN KEY (user_id) REFERENCES `user`(user_id)
        ON UPDATE CASCADE ON DELETE SET NULL,
    CONSTRAINT fk_vid_kat
        FOREIGN KEY (kategorie_id) REFERENCES kategorie(kategorie_id)
        ON UPDATE CASCADE ON DELETE SET NULL
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;

-- ---------- bilder ----------
CREATE TABLE bilder (
    bild_id        INT AUTO_INCREMENT PRIMARY KEY,
    vid_id         INT           DEFAULT NULL,
    kategorie_id   INT           DEFAULT NULL,
    dateipfad      VARCHAR(500) NOT NULL,
    beschreibung   TEXT,
    laufzeit       INT           DEFAULT 0,
    hochgeladen_am DATETIME      NOT NULL DEFAULT CURRENT_TIMESTAMP,
    CONSTRAINT fk_bild_vid
        FOREIGN KEY (vid_id) REFERENCES videos(vid_id)
        ON UPDATE CASCADE ON DELETE SET NULL,
    CONSTRAINT fk_bild_kat
        FOREIGN KEY (kategorie_id) REFERENCES kategorie(kategorie_id)
        ON UPDATE CASCADE ON DELETE SET NULL
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;
```

```
MariaDB [test]> SHOW INDEX FROM kunde;
+-------+------------+------------------+--------------+-------------+-----------+-------------+----------+--------+------+--------
----+---------+---------------+---------+
| Table | Non_unique | Key_name         | Seq_in_index | Column_name | Collation | Cardinality | Sub_part | Packed | Null | Index_t
ype | Comment | Index_comment | Ignored |
+-------+------------+------------------+--------------+-------------+-----------+-------------+----------+--------+------+--------
----+---------+---------------+---------+
| kunde |          0 | PRIMARY          |            1 | id          | A         |          50 |     NULL | NULL   |      | BTREE
    |         |               | NO      |
| kunde |          0 | idx_email_unique |            1 | email       | A         |          50 |     NULL | NULL   | YES  | BTREE
    |         |               | NO      |
| kunde |          1 | idx_name         |            1 | name        | A         |          50 |     NULL | NULL   | YES  | BTREE
    |         |               | NO      |
| kunde |          1 | idx_name_telefon |            1 | name        | A         |          50 |     NULL | NULL   | YES  | BTREE
    |         |               | NO      |
| kunde |          1 | idx_name_telefon |            2 | telefon     | A         |          50 |     NULL | NULL   | YES  | BTREE
    |         |               | NO      |
+-------+------------+------------------+--------------+-------------+-----------+-------------+----------+--------+------+--------
----+---------+---------------+---------+
5 rows in set (0.009 sec)
```

Zunächst sieht man den PRIMARY KEY, der auf der Spalte *id* liegt. Dieser Index ist eindeutig und sorgt dafür, dass jeder Datensatz eindeutig identifiziert werden kann. Zusätzlich wurde ein UNIQUE Index auf die Spalte email erstellt. Dieser verhindert doppelte Einträge und verbessert gleichzeitig die Suchgeschwindigkeit bei Abfragen über die E-Mail-Adresse.
Außerdem existiert ein normaler Index auf der Spalte *name*, der die Suche nach Namen beschleunigt, jedoch doppelte Werte erlaubt.
Ein weiterer wichtiger Index ist der kombinierte Index auf name und telefon. Dieser besteht aus zwei Spalten und wird besonders effizient genutzt, wenn Abfragen beide Spalten oder zumindest die erste Spalte (*name*) enthalten. Die Reihenfolge der Spalten spielt dabei eine entscheidende Rolle.
Die Spalte *Non_unique* zeigt an, ob ein Index eindeutige Werte erzwingt, während *Seq_in_index* die Reihenfolge der Spalten innerhalb eines Indexes angibt. Die *Cardinality* gibt an, wie viele unterschiedliche Werte ungefähr vorhanden sind.
Zusammenfassend lässt sich sagen, dass Indizes die Performance von Datenbankabfragen deutlich verbessern, jedoch gezielt eingesetzt werden sollten, da zu viele oder doppelte Indizes unnötigen Speicher verbrauchen und Schreiboperationen verlangsamen können.

Resultat

Im Rahmen der praktischen Arbeit wurde eine Media-Hosting-Plattform erfolgreich umgesetzt. Die Webseite ermöglicht es, verschiedene Medientypen wie Bilder, Videos, Audio und Texte darzustellen und zu verwalten.

Die Benutzeroberfläche zeigt die Inhalte strukturiert in verschiedenen Bereichen an. Die Navigation über Kategorien und Medientypen funktioniert über eine Seitenleiste (Sidebar) und erlaubt dem Benutzer, schnell auf unterschiedliche Medien zuzugreifen.

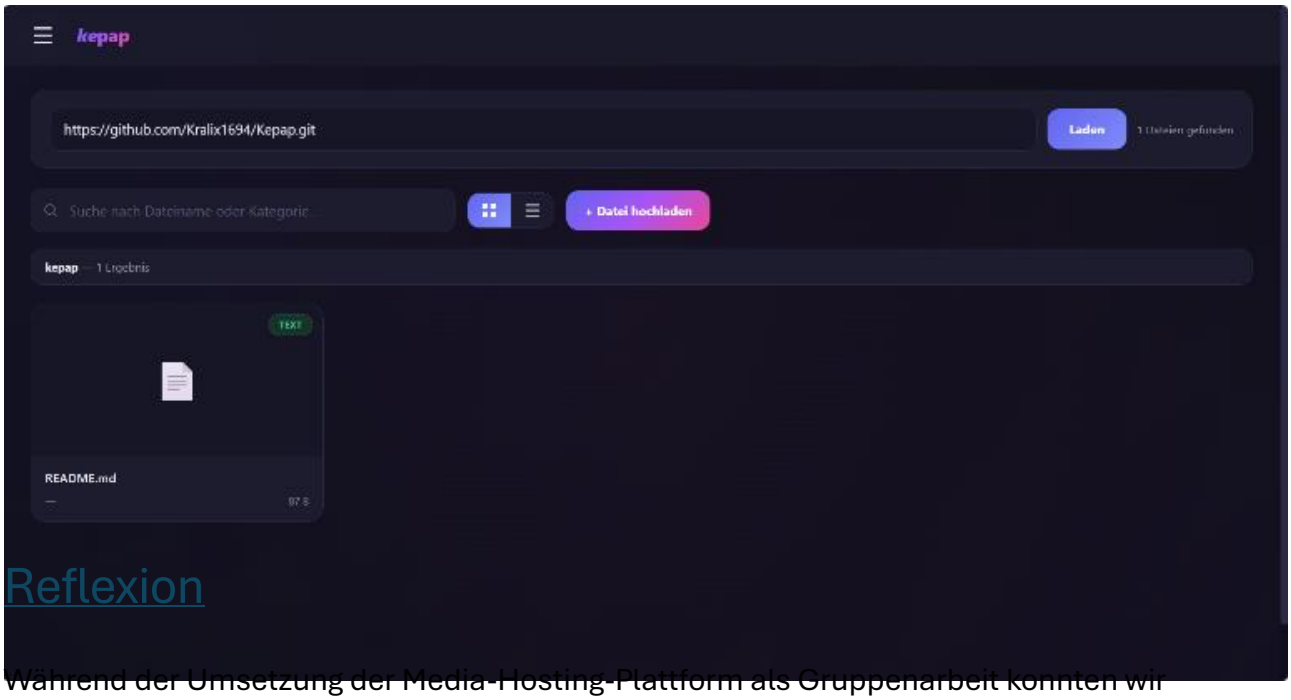
Die Darstellung der Inhalte erfolgt visuell innerhalb der Webseite, wobei mehrere Elemente gleichzeitig angezeigt werden können. Dadurch entsteht eine dynamische Oberfläche, in der verschiedene Medienbereiche kombiniert dargestellt werden.

Zusätzlich konnten grundlegende Funktionen wie:

- Navigation über Sidebar mit Filter nach Typ und Kategorie
- Strukturierte Anzeige von Medien (Raster- und Listenansicht)
- Volltextsuche nach Dateinamen und Kategorie
- Datei-Upload und Löschen über die GitHub API
- Verbindung zum GitHub-Repository als Datenspeicher

erfolgreich implementiert werden.

Insgesamt erfüllt die Anwendung die grundlegenden Anforderungen einer Media-Hosting-Plattform und zeigt die praktische Umsetzung der geplanten Funktionen.



Reflexion

Während der Umsetzung der Media-Hosting-Plattform als Gruppenarbeit konnten wir viele neue Erfahrungen sammeln. Besonders das Zusammenspiel von Frontend (HTML, CSS, JavaScript) und der GitHub API als Datenspeicher war für uns sehr lehrreich.

In der Gruppe konnten wir die Aufgaben aufteilen und gemeinsam Lösungen erarbeiten. Dadurch konnten wir effizienter arbeiten und voneinander lernen. Jeder konnte seine Stärken einbringen, zum Beispiel im Bereich Design, Programmierung oder Datenbank.

Eine Herausforderung war die Abstimmung innerhalb der Gruppe. Es war wichtig, dass alle am gleichen Stand arbeiten und die einzelnen Teile zusammenpassen. Auch beim Verbinden von GitHub API und Webseite traten Schwierigkeiten auf, die wir gemeinsam lösen konnten.

Positiv war die Zusammenarbeit im Team. Durch gegenseitige Unterstützung konnten Probleme schneller erkannt und behoben werden.

Rückblickend würden wir beim nächsten Mal die Aufgaben noch klarer aufteilen und die Kommunikation in der Gruppe verbessern, damit die Zusammenarbeit noch effizienter wird.

Insgesamt war die Gruppenarbeit sehr lehrreich und hat uns geholfen, sowohl unsere technischen Fähigkeiten als auch unsere Teamarbeit zu verbessern.